INF-2700: Database systems

Assignment 3

Jens Christian Valen Leynse
Jle040
jle040@post.uit.no

Nov 08th, 2019

Introduction

The assignment is separated into 3 parts. Part 1 is implementing a natural join operation with both a tuple- and a block-based design. This part assumes that the two tables only has a single common integer attribute. The next part is about the performance of the implementation. By making the program handle large tables the objective is to profile and compare the performance of the two algorithms. Lastly a written-only part is included where the goal is to do a thought experiment comparing the block nested-loop join algorithm with an algorithm that makes use of the B^+-tree file organizations.

Design & Implementation

Task 1: Implementing natural join

The natural join functions goal is to combine two tables into one, combining fields with the same data. So, the function starts by creating a new schema and as a debugging feature chooses between joining on a tuple-based algorithm or a block-based algorithm. Next the two given schemas are sent together with the new schema to the combine fields function.

The combine fields function takes the fields from both given tables and puts them into the new schema. It does this by first creating copies of both of them to avoid any pointer issues. Next a double loop iterates though both finding every field that is the same in both given tables. The common fields found have their names stored in an array. After resetting the pointers of the fields in both tables, a loop goes though both of them again and checks if their name was saved during the previous process. If it hasn't been saved, the field is added to the new schema. The loop also makes sure to also add the field saved when it is found in one of the checks exclusively.

Next by getting back to the natural join function the array with the field names is iterated though. When both the field pointer to the left table, and the field pointer to the right table of the tables given the function is pointing towards the saved field, the next function which finds the records is called.

The first looping function uses the tuple-based nested-loop join algorithm. This function starts by creating new records for both tables as well as get the page for the next record in the

schema. The following loops then start by going though the left pages, finding the position and the values of this position given the offset of the field they both have in common. The loop then takes out the record for the later merge with the right table. The next loop for the right table does the same in that it iterates through the pages. It also makes sure to not run without data after the end of the page has been met. Next the position and its value are retrieved, and the value is compared with the value from the left position given the left page. If they are the same, the fields have overlapping data and the records can merge. So, the record from the right page is retrieved sent to be merged with the left record and appended to the resulting schema. When this value is found the innermost loop is also broken since the process assumes only one record has the same value. Until the value is found the position is continuously updated. Lastly the schema with all of its newly added records are returned to the natural join function again.

The looping function that uses the block-based nested-loop join algorithm contains much of the same code as the tuple-based function but differs in that it checks though the pages given the block number. So, the blocks for both the right and the left table is iterated though before the next loop which adds one record length to the position given the page for each iteration. Otherwise it also takes the values at the current position, updates the position through the iterations and merges the records when the value is found. Since the block-based loops uses the block number for the pages, the pages are unpinned at the end of each iteration. The resulting schema is then returned to the natural join function which directly returns it as a table to the database.

The function that merges the records is given all of the schemas and records when it's called. It mimics the combine fields function by copying the schemas, then it defines the field pointers as well as initializes the counter. The counters help keep the current field position in all three schemas as numbers easy to read to the records. The next loop goes through the predetermined fields in the resulting schema. Since the field name has already been given, the function can do a quick check to see which of the schemas contain the field at the pointer position. When a field is found the data is added by copying the memory, and the field pointer for the source schema is reset to zero while the resulting schema goes to the next field. By using this way, it also ensures that no field triggers both checks since the field position is moved after having been found. All of the temporary schemas are removed afterwards, since they no longer have any use.

Task 3: Performance

Unfortunately for the sake of profiling the performance it's impossible with the current setup. The process can be ran giving the correct resulting table. But some unforeseen issues hinder the program from running the test files. Whenever such an attempt is made the program either responds with a double free/ corruption message or a segmentation fault.

Task 4: Think out of the box

Another join algorithm used in the implementation of database management systems that can make use of the B^+-tree file organization system is the sort-merge join algorithm. This is based on the fact that the algorithm needs both inputs to be presented in sorted order. When the input produced is tree based, this fits the criteria as the appropriate keys are provided.

The main basis of the algorithm is to sort by the join attribute, which will result with linear scans that are interleaved, being able to encounter the sets given the attributes at the same time. The algorithm can be described simply as the case of an inner join of two relations on a single attribute

The algorithm block nested-loop when it comes to O notation runs in either $O(P_r P_s/M)$ I/Os, where P_s and P_r are sizes of the join operands in pages, and M is the available pages of internal memory. Or $O(P_r + P_s)$ I/Os if the outer join operand fits in the available internal memory.

The sort-merge join algorithm in comparison has the notation $O(P_r + P_s)$ I/Os in the worst case. While if the join operands aren't sorted it can cost additional sorting time equal $O(P_r \log(P_r) + P_s \log(P_s))$ I/Os.

Discussion

There are several issues that came up during debugging at the later stages. The most notable ones being the fact that the database receives three new tables instead of one. I was made aware of this during the last day of debugging and the fault lies in the copy schema functions in the combine fields function. They can't be removed afterwards as the stops the whole program from running, and by debugging the different schemas here they somehow acquire five empty fields while inside the function, and one of them changes its name to the first field name. As the copy functions were implemented to prevent an earlier issue with fields being added to the wrong schema the functions can't be removed without bringing this problem back, and there wasn't enough time after the issue was discovered to create a new function or find a work around.

The next issue is contained in the block nested loop join function. This function is at the moment of assignment delivery not able to return the entire correct table as it only returns the records from the first block. The issue is believed to be coming from the fact that the right block skips one position for each time the left block switches.

Conclusion

The program is able to take to tables given and merge them by using natural join operations. The resulting table from the nested loop join algorithm is retrieved with all records while some issues still retain in the block nested loop join algorithm hindering the output of more than one block of data. The performance can't be tested given the test functions as profiling won't return any data. Lastly the out of the box task has been completed taking into consideration a sorting algorithm that takes advantage of a B^+-tree file organization style, and comparing the two.